

Xi

DATASHEET + MANUAL TÉCNICO

Asistente robótica jRobot basada en ESP32-S3, XiaoZhi, voz local y modelos de lenguaje

Estado documentado	Prototipo estable + puente local de voz
Hardware	ESP32-S3 / OLED / micrófono I ² S / altavoz I ² S / WS2812
Cerebro local validado	Qwen3.5 4B servido mediante Ollama
Reconocimiento de voz	Silero VAD + Faster-Whisper small
Voz	Edge TTS, es-CL-CatalinaNeural
Transporte	Wi-Fi + WebSocket + Opus
Firmware de esta rama	ESP-IDF 5.5.4

Objetivo: poder explicar qué es Xi, qué hay detrás de su voz, qué hace cada componente, cómo se conecta a una red, qué vive en el ESP32 y qué vive en el PC, qué recuerda hoy y cómo puede evolucionar hacia una sola Xi con múltiples cuerpos.

Versión: 17 de agosto de 2026



0. Mapa rápido

#	Sección
1	Xi en una frase
2	Arquitectura completa
3	Hardware y pinout
4	Cómo viaja una conversación
5	Diccionario técnico
6	Ollama y Qwen
7	Whisper, VAD y TTS
8	Red, WebSocket, Opus y OTA
9	Cambiar de Wi-Fi
10	Modos Ollama / GPT / XiaoZhi
11	Qué vive en el PC y qué vive en Xi
12	Memoria
13	Una Xi, muchos cuerpos
14	HUB, MQTT, MCP y RAG
15	Instalación en otro PC
16	Diagnóstico
17	Cómo explicarla
18	Fuentes y límites

Idea que conviene recordar desde el principio: Xi no es “un programa”. Es una cadena formada por hardware, firmware, red, servidor, reconocimiento de voz, modelo de lenguaje y síntesis de voz.

1. Xi en una frase

Xi es un terminal robótico de voz. El ESP32-S3 escucha, muestra su rostro, reproduce audio y mantiene la conexión; el razonamiento pesado puede ejecutarse en un PC local o en un servicio remoto.

La placa no ejecuta Qwen3.5 4B. Su trabajo principal es ser el cuerpo de interacción: micrófono, altavoz, pantalla, LED, Wi-Fi y protocolo. El PC puede actuar como cerebro superior.

Metáfora útil

Parte	En Xi	Analogía
ESP32-S3	control físico y comunicaciones	sistema nervioso periférico
Micrófono + VAD + Whisper	captura y transcripción	oído
Ollama + Qwen	razonamiento y texto	cerebro lingüístico
TTS	texto a voz	voz
OLED + LED	rostro y estado	expresión
WebSocket + Opus	canal de datos/audio	nervios
HUB / memoria futura	estado, recuerdos, documentos	memoria y contexto

Nota de nombres: el repositorio conserva XiaoZhi como base técnica. La personalización jRobot se llama Xi; en una rama/manual actual también aparece Shi como nombre hablado. Cambiar el nombre conversacional no cambia la arquitectura.

2. Arquitectura completa

Cadena local validada en el proyecto:



- Xi captura audio con el micrófono I²S.
- Silero VAD decide cuándo hay voz y cuándo termina el turno.
- Faster-Whisper convierte la voz en texto.
- El servidor envía ese texto a Ollama.
- Ollama ejecuta Qwen3.5 4B y devuelve texto.
- Edge TTS sintetiza una voz femenina chilena.
- Opus comprime el audio; WebSocket lo devuelve a Xi.
- El ESP32 reproduce la voz y anima la boca según el PCM realmente reproducido.

Conclusión: la “personalidad” que vemos al hablar con Xi emerge de la cooperación entre varios módulos. Ninguno de ellos, por separado, es Xi completa.

3. Hardware y pinout

Componente	Función	Dato actual
ESP32-S3	Control principal	Flash 16 MB / PSRAM 8 MB
OLED SSD1306	Rostro y estados	128 × 64
Micrófono I ² S	Entrada de voz	16 kHz en servidor actual
Altavoz I ² S	Salida de voz	24 kHz documentado
WS2812	LED de estado	GPIO 48
Wi-Fi	Red	2,4 GHz
CH343	USB/serial	COM variable

GPIO validados

Función	GPIO
I ² S BCLK	4
I ² S WS/LRCK	5
Altavoz DOUT	6
Micrófono DIN	7
OLED SDA / SCL	8 / 9
Lámpara MCP prueba	18
BOOT	0
Touch declarado	47
Volumen - / +	39 / 40
WS2812	48

Servos futuros: yaw/pitch de cabeza, dos brazos y antena. Alimentación separada 5-6 V, tierra común y preferiblemente PCA9685. No alimentar cinco servos desde el regulador de la placa.



4. Cómo viaja una conversación

A. Escucha

El micrófono entrega muestras digitales al ESP32 mediante I²S y la placa transmite audio al servidor.

B. Detecta voz

Silero VAD separa voz útil de silencio/ruido y ayuda a cerrar el turno.

C. Transcribe

Faster-Whisper interpreta el audio y produce texto en español.

D. Razona

El servidor envía el texto a Ollama; Ollama ejecuta Qwen3.5 4B.

E. Sintetiza

Edge TTS genera audio con la voz es-CL-CatalinaNeural.

F. Comprime y envía

Opus comprime el audio y WebSocket lo devuelve a Xi.

G. Habla y expresa

El altavoz reproduce la respuesta; la boca se anima con la energía PCM realmente reproducida.

Latencia observada en la configuración documentada: Qwen caliente ~0,9-1,5 s; Whisper suele añadir alrededor de 2 s. La primera carga de Qwen tras reiniciar el PC puede ser bastante más lenta.

5. Diccionario técnico

Término	Qué significa dentro de Xi
ESP32-S3	Microcontrolador que ejecuta firmware y maneja pantalla, audio, LED, Wi-Fi y periféricos.
Firmware	Programa grabado dentro del ESP32.
ESP-IDF	SDK oficial de Espressif para compilar firmware.
I²S	Bus digital especializado en audio.
PCM	Audio digital sin comprimir representado como muestras numéricas.
VAD	Voice Activity Detection: decide si hay voz o silencio.
ASR / STT	Reconocimiento de voz / Speech-to-Text.
Whisper	Familia de modelos de reconocimiento de voz.
Faster-Whisper	Implementación optimizada de Whisper con CTranslate2.
LLM	Large Language Model: modelo que comprende y genera texto.
Qwen	Familia de modelos; Xi usa qwen3.5:4b en el prototipo local.
Ollama	Runtime/servidor para descargar, ejecutar y exponer modelos mediante API.
TTS	Text-to-Speech: convertir texto en voz.
Edge TTS	Servicio online usado para la voz actual.
Opus	Codec de audio eficiente para transmisión.
WebSocket	Conexión persistente bidireccional entre Xi y el servidor.
OTA	Over-The-Air: configuración/actualización por red.
API	Interfaz para que un programa hable con otro.
Python venv	Entorno aislado para dependencias Python.
FFmpeg / libopus	Procesamiento, conversión y codec de audio.
MQTT	Protocolo publish/subscribe ideal para telemetría.
MCP	Protocolo para exponer herramientas estructuradas al modelo.
RAG	Buscar documentos relevantes antes de responder.
Git / GitHub	Versionado local y repositorio remoto/sincronización.

6. Ollama y Qwen

Ollama no es la inteligencia de Xi. Ollama es el programa que administra y ejecuta modelos. Qwen es el modelo que genera la respuesta.

Piensa en Ollama como el motor y Qwen como el cerebro que instalas dentro del motor. El servidor de Xi llama a la API local de Ollama.

En una instalación estándar, Ollama sirve su API local en **http://localhost:11434**.

Qwen3.5:4b

Es un modelo compacto de la familia Qwen3.5. El puente actual lo ejecuta localmente para reducir dependencia de APIs externas y mantener una latencia razonable.

Ollama hace

- descargar modelos (**ollama pull**);
- cargarlos en CPU/GPU;
- administrar su ejecución;
- exponer una API;
- permitir cambiar de modelo sin reescribir Xi.

Ollama no hace

- escuchar el micrófono;
- transcribir voz;
- generar directamente la voz del altavoz;
- ser la memoria persistente total de Xi;
- sincronizar proyectos entre computadores.

```
ollama pull qwen3.5:4b
```




7. Whisper, VAD y TTS

Silero VAD

Es el detector de actividad de voz. Ayuda a saber cuándo empezaste y terminaste de hablar, evitando cortes absurdos o silencios eternos.

Faster-Whisper

Es el oído lingüístico. Convierte audio en texto. La implementación Faster-Whisper usa CTranslate2 y admite cuantización; Xi está documentada con modelo **small** en CPU/INT8.

TTS

Text-to-Speech convierte el texto final en audio. El sistema actual usa **Edge TTS** con la voz **es-CL-CatalinaNeural**.

Arquitectura híbrida: el LLM puede funcionar localmente, pero Edge TTS todavía requiere Internet. Una evolución lógica es reemplazarlo por Piper u otro TTS local para operar completamente offline.

Opus

El PCM sin comprimir pesa mucho. Opus reduce el ancho de banda manteniendo buena calidad y baja latencia para voz.

8. Red, WebSocket, Opus y OTA

Wi-Fi: conecta Xi con el PC/servidor o con Internet. El prototipo actual trabaja en 2,4 GHz.

WebSocket: mantiene una conexión abierta en ambos sentidos para mensajes y audio continuos.

Opus: comprime el audio para transmitirlo.

OTA: permite entregar información/configuración por red y forma parte del mecanismo de arranque/conexión.

Direcciones validadas en la instalación original

```
WebSocket: ws://192.168.1.89:8000/xiaozhi/v1/  
OTA local: http://192.168.1.89:8003/xiaozhi/ota/
```

192.168.1.89 no es una identidad permanente. Era la IP del PC en esa red. En otra casa, oficina o hotspot puede cambiar. A futuro conviene usar hostname, reserva DHCP, descubrimiento local o una configuración dinámica.

El firmware del puente intenta usar el servicio local durante el arranque y conserva una salida hacia el servicio oficial XiaoZhi si el puente local no está disponible.

9. Cómo conectar Xi a otra red Wi-Fi

La base XiaoZhi soporta provisión de Wi-Fi mediante hotspot/punto de acceso. Las credenciales quedan guardadas en memoria no volátil del ESP32.

Caso A: repetir nombre y clave

Si el celular o un router crea una red **2,4 GHz** con exactamente el mismo SSID y contraseña que Xi ya conoce, normalmente podrá conectarse sin reprogramar, siempre que el tipo de seguridad sea compatible.

Caso B: red nueva

1. Poner Xi en modo de configuración Wi-Fi o dejar que entre en provisión cuando no encuentre una red conocida.
2. Xi crea un hotspot de configuración.
3. Conectar el teléfono/PC a ese hotspot.
4. Abrir el portal cautivo.
5. Elegir la red real e introducir la contraseña.
6. Xi guarda las credenciales y vuelve a conectarse.

jRobot S3: la base XiaoZhi soporta provisión por hotspot, pero la combinación exacta para forzar manualmente el modo de configuración debe verificarse en el firmware instalado. BOOT se usa para conversación una vez iniciado; no conviene inventar una combinación de botones no probada.

PC y Xi en redes diferentes

Una IP privada 192.168.x.x no cruza Internet. Para usar el PC de casa desde la oficina necesitaremos una capa segura: VPN/overlay o túnel autenticado. **No abrir Ollama o el bridge directamente a Internet sin autenticación.**

10. Modos Ollama / GPT / XiaoZhi

Modo	Cerebro	PC	Internet	LLM
Ollama	qwen3.5:4b local	Sí	Solo TTS actual	sin tokens OpenAI
GPT	OpenAI API	Sí	Sí	consume API
XiaoZhi	Nube oficial	No tras configurar	Sí	según servicio

La conmutación ocurre al iniciar/reiniciar Xi, no a mitad de una respuesta.

Selector documentado

```
Set-XiMode.ps1 -Mode Ollama
Set-XiMode.ps1 -Mode XiaoZhi
Set-XiMode.ps1 -Mode GPT
Set-XiMode.ps1 -Mode Status
```

La clave de OpenAI se guarda fuera del repositorio. El modo GPT no hereda automáticamente conversaciones o memoria de Codex/ChatGPT.

11. Qué vive en el PC y qué vive en Xi

Xi / ESP32

- firmware ESP-IDF;
- OLED y expresiones;
- captura/reproducción I²S;
- LED y estados;
- Wi-Fi y protocolo;
- Opus / WebSocket;
- BOOT y periféricos.

PC local

- servidor Python;
- Silero VAD;
- Faster-Whisper;
- Ollama;
- Qwen3.5:4b;
- Edge TTS;
- FFmpeg / libopus;
- scripts y configuración.

Por eso “clonar Xi” no basta. GitHub puede traer el código, pero en un PC nuevo también hay que instalar runtime, modelos, dependencias y configuración.

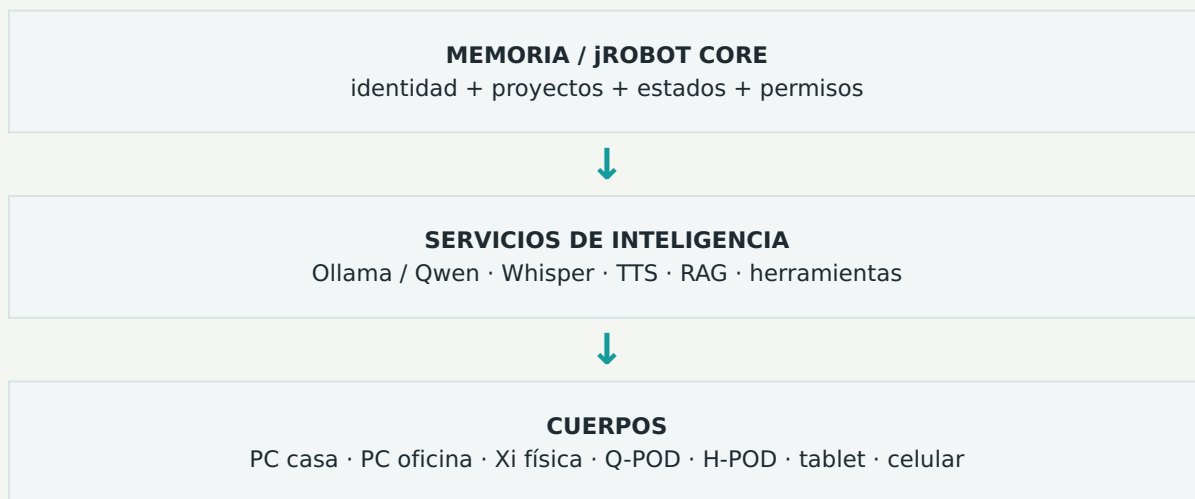
12. Memoria: qué recuerda hoy

Ollama/Qwen no es una memoria central permanente. El modelo conserva conocimiento de entrenamiento y puede recibir historial de conversación, pero los recuerdos personales y de proyectos deben almacenarse explícitamente si queremos compartirlos entre casa, oficina y robots.

Tipo	Qué guarda	Ejemplo
Documental	docs, pinouts, BOM, firmware, decisiones	"Mini Q-POD usa MG92B"
Estable	identidad y decisiones vigentes	"Xi habla español"
En vivo	datos con fecha que caducan	"batería 11,4 V a las 14:03"

Meta: separar identidad y memoria del computador. Entonces la Xi de casa y la de oficina pueden ser dos terminales que consultan la misma memoria, no dos "gemelas" que se van separando con el tiempo.

13. Una Xi, muchos cuerpos



Arquitectura recomendada: edge + server

Cada cuerpo mantiene funciones básicas locales y seguras; el Core entrega memoria e inteligencia superior cuando hay conexión.

Un robot debe poder detenerse y proteger sus motores aunque Internet falle. Una pregunta compleja sobre la historia de un proyecto puede consultar el Core.

14. HUB, MQTT, MCP y RAG

MQTT

Ideal para telemetría frecuente de robots:

```
robots/{robot_id}/state
robots/{robot_id}/battery
robots/{robot_id}/mode
robots/{robot_id}/alerts
robots/{robot_id}/telemetry/{sensor}
robots/{robot_id}/commands/{command}
```

RAG

Xi buscará documentación actual antes de responder. No es necesario reentrenar Qwen cada vez que cambia un pinout.

MCP

Permitirá herramientas estructuradas como **robots.get_battery()** o **projects.get_pinout()**.

HUB

Normalizará datos, guardará historial, mostrará alertas y dará a Xi un estado actual verificable.

Órdenes físicas: consultar puede ser solo lectura. Mover motores o actuadores debe pasar por validación, límites y confirmaciones. Parar un robot puede ser una excepción segura.

15. Instalar Xi en otro PC

La meta ideal: **clonar repos** → **instalador** → **descargar modelos** → **configurar red** → **Xi lista**. Hoy todavía hay pasos manuales.

Hay que replicar

- Jcabroc/xiaozhi-xi;
- Jcabroc/xiaozhi-esp32-server;
- Python 3.10 + entorno virtual;
- dependencias Python;
- Ollama;
- qwen3.5:4b;
- FFmpeg + libopus;
- configuración local del bridge;
- ESP-IDF 5.5.4 si ese PC compilará/flashearé esta rama.

Comandos base

```
git clone https://github.com/Jcabroc/xiaozhi-xi.git

git clone https://github.com/Jcabroc/xiaozhi-esp32-server.git

winget install Ollama.Ollama
ollama pull qwen3.5:4b
```

No subir a GitHub: claves API, configuración privada, grabaciones, contraseñas ni logs sensibles.

16. Diagnóstico y mantenimiento

Síntoma	Primero revisar
LED verde, no responde	modo, servidor, OTA y que PC/Xi compartan LAN
Transcribe absurdo	ruido, orientación del micrófono, WAV temporales
Tarda mucho	Qwen precalentado, razonamiento no excesivo
Idioma/voz incorrecta	prompt y voz TTS
No aparece COM	cable de datos, CH343, reconexión
PC nuevo no encuentra Xi	firewall, IP, 2,4 GHz y bridge
Reinicios con servos	fuelle separada, GND común, capacitor, ruido
Casa sí, oficina no	IP distinta o redes LAN distintas

Regla de oro: separar la cadena. ¿Micrófono capta? ¿VAD detecta? ¿Whisper transcribe? ¿Ollama responde? ¿TTS genera audio? ¿WebSocket entrega? ¿Altavoz reproduce? Cuando pruebas bloque por bloque, la magia negra se transforma en ingeniería.

17. Cómo explicar Xi

En 20 segundos

“Xi es una asistente robótica basada en ESP32-S3. El dispositivo maneja micrófono, altavoz, pantalla y Wi-Fi; un PC puede hacer el procesamiento pesado: Whisper convierte mi voz a texto, Qwen corre localmente mediante Ollama y TTS convierte la respuesta otra vez en voz. Todo se comunica por WebSocket con audio Opus.”

En dos minutos

Xi no es solo un chatbot dentro de una placa. La placa es su cuerpo: escucha por I²S, muestra un rostro OLED, usa LED de estado y se conecta por Wi-Fi. El audio llega al servidor local; VAD detecta el turno, Faster-Whisper transcribe y Ollama ejecuta Qwen3.5 4B. Luego Edge TTS genera una voz chilena y el audio vuelve comprimido con Opus por WebSocket. También puede usar GPT o el servicio oficial XiaoZhi. La siguiente etapa es separar memoria e identidad del PC para que la misma Xi pueda existir en casa, oficina y robots mediante un jRobot Core.

No confundir

- Ollama no es “la IA”: ejecuta el modelo.
- El ESP32 no realiza todo el razonamiento pesado.
- Xi todavía no tiene una memoria central total y compartida.
- El sistema actual no es completamente offline porque Edge TTS requiere Internet.

18. Fuentes y límites

Proyecto jRobot consultado

- Jcabroc/xiaozhi-xi, rama agent/xi-voice-bridge;
- README_XI.md;
- docs/MANUAL_SHI.md;
- bridge/README.md;
- docs/ARQUITECTURA_HUB.md;
- PR “Add Xi local Ollama voice bridge”;
- Jcabroc/xiaozhi-esp32-server, PR “Add Spanish local Whisper and Ollama support”.

Fuentes técnicas externas

- Ollama official documentation;
- Qwen Team / Qwen3.5-4B model card;
- SYSTRAN faster-whisper;
- rany2/edge-tts;
- 78/xiaozhi-esp32.

Estado: este documento describe el prototipo/rama verificado en agosto de 2026. Distingue piezas actuales de arquitectura futura. Si el firmware o el bridge cambian, el datasheet debe versionarse.

Mejora recomendada

Crear dentro del repositorio un **XI_SYSTEM_MANIFEST.json** con versión de firmware, modelo LLM, ASR, TTS, hostname/IP, puertos y hashes de configuración. Así siempre podremos responder “¿qué versión exacta de Xi está corriendo aquí?”.

jRobot · Xi · agosto 2026

Xi ElectronBot Body v0.3

Manual de integración hardware, voz emocional, pines, escalabilidad y plan para Codex

Proyecto jRobot / Xi / XiaoZhi / Kira Core S3 - 2026-08-27 - actualización batería

Este documento actualiza la versión 0.2 y agrega la nueva etapa de Xi: cuerpo ElectronBot impreso en resina, seis servos, IMU, PIR, botones físicos externos, sensores touch, voz emocional y reserva de pines para pantalla/cara verde futura.

Resumen de decisión

Xi evolucionará desde placa/tupper hacia un cuerpo inspirado en ElectronBot. El cuerpo se usará principalmente como mecánica y proporciones; el alma será jRobot: XiaoZhi/Xi, servidor local, HUB, memoria, voz expresiva, cara OLED anime, sensores y servos con seguridad.

- Lo inmediato: guardar una versión estable y hacer pruebas simples con alimentación externa o powerbank.
- Lo siguiente: agregar cara OLED anime, touch para activar escucha, IMU, PIR y servos usando una placa PCA9685.
- Lo avanzado: servidor local xiaozhi-esp32-server, voz expresiva tipo Elevenlabs v3 o TTS local, memoria central Xi y conexión al HUB.
- Batería: aún queda abierta. Se debe medir consumo real antes de decidir LiPo interna definitiva.
- Estado de batería: agregar medición VBAT por ADC para que Xi pueda cambiar expresiones, voz y conducta según carga baja, carga crítica o energía estable.

1. Voz emocional: Elevenlabs v3, voz local y voz Nya/Xi

La voz actual de XiaoZhi se siente más viva porque su flujo de voz está pensado para conversación hablada. La Xi del PC se siente más plana porque probablemente usa un TTS menos expresivo y no tiene una capa de dirección emocional que coordine texto, voz, cara y cuerpo.

Elevenlabs v3 es una referencia útil porque permite audio tags, es decir, marcas como risa, susurro, tristeza, enojo o suspiro que cambian la actuación de la voz. Para Xi esto se debe tratar como una opción de modo pro/demostración, no como única dependencia vital.

Opción	Ventaja	Desventaja	Uso recomendado
Elevenlabs v3	Voz muy expresiva, tags emocionales, buena calidad.	Servicio cloud/pago; dependencia externa.	Demos, escenas, voz premium de Xi/Nya.
XiaoZhi cloud/server	Ya funciona con la S3, conversación fluida.	Puede depender de nube o configuración del server.	Prototipo operativo y transición a servidor propio.
TTS local	Control local, privacidad, menor dependencia.	Más trabajo; calidad variable; requiere entrenamiento/ajuste.	Objetivo jRobot a mediano plazo.
Voz Nya/Xi propia	Identidad única para universo Nya y Xi.	Requiere dataset, permisos, pruebas y entrenamiento.	Voz oficial compartida entre Nya y Xi.

Costos orientativos de Elevenlabs consultados el 2026-08-27

Los precios cambian, por lo que deben verificarse antes de contratar. En la página oficial de Elevenlabs API se muestran planes como Free, Starter, Creator, Pro, Scale y Business. Como referencia, se observó Free con créditos incluidos, Starter a 6 USD/mes, Creator a 22 USD/mes, Pro a 99 USD/mes, Scale a 299 USD/mes y Business a 990 USD/mes. En API, TTS v3 aparece alrededor de 0.10 USD por 1.000 caracteres, y v3 Conversational alrededor de 0.05 USD por 1.000 caracteres. Fuente: elevenlabs.io/pricing/api y elevenlabs.io/pricing.

Estrategia recomendada para la voz Nya/Xi

1. No copiar una voz de terceros. Crear una voz propia para Nya/Xi con material autorizado o generado por nosotros.
2. Usar Elevenlabs v3 para prototipos emocionales y evaluar cuánto cuesta realmente con frases cortas.
3. En paralelo estudiar TTS local compatible con xiaozhi-esp32-server: FishSpeech, GPT-SolVITS, Index-TTS, PaddleSpeech u opciones equivalentes.
4. Crear Xi Emotion Director: una capa que convierte intención en emoción, texto, tags de voz, cara OLED y movimiento de servos.
5. Unificar memoria/persona: Xi PC y Xi S3 deben consultar la misma memoria central para no parecer dos almas distintas.

2. Materiales disponibles y pendientes

Categoría	Material	Estado	Notas
Cuerpo	Cuerpo ElectronBot impreso en resina	Disponible	Usar proporciones originales; adaptar electrónica sin modificar demasiado.
Servos	10 micro servos tipo ElectronBot	Disponible	Usar 6 en Xi; no modificarlos en primera etapa.
MCU	ESP32-S3 N16R8 / Kira Core S3	Disponible	Ya probado con XiaoZhi, OLED, audio, Wifi y activación.
Pantalla actual	OLED SSD1306 128x64 I2C	Disponible	Cara anime inicial. Color actual puede ser azul; objetivo futuro verde.
Audio entrada	INMP441 I2S	Disponible	Microfono principal.
Audio salida	MAH98357A + parlante	Disponible	Bajar volumen para evitar saturación. SD del módulo a 3.3V.
Sensores	IMU	Disponible	Preferible I2C compartido con OLED.
Sensores	Mini PIR	Disponible	Detectar presencia/movimiento frente a Xi.
Interacción	2 botones físicos externos	Disponible	Reset y BOOT/escucha porque la ESP quedará oculta.
Interacción	Touch capacitivo mínimo 4 pads	Por instalar	Cabeza, izquierda, derecha, cuerpo/base.
Expresión	Nueva cara verde / pantalla futura	Reservar pines	Mantener OLED actual y reservar SPI para upgrade.
Visión	Cámara	Pendiente	Agregar después de estabilizar cuerpo, energía y servos.
Energía	Powerbank 20.000 mAh Xiaomi/Redmi	Disponible	Ideal para pruebas de mesa. Revisar salida 5V 2A/3A.
Energía	LiPo 1S 2000-5000 mAh	Pendiente	Elegir después de medir consumo real.
Sensores / energía	Medición de batería por ADC	Agregar	Divisor resistivo desde VBAT a ADC1; permite expresiones por batería y avisos de carga.

3. Datasheet de pines propuesto - Xi ElectronBot S3

Objetivo: usar pocos pines directos de la ESP32-S3 y concentrar servos por I2C mediante PCA9685. Esto deja margen para touch, PIR y futura pantalla. Confirmar siempre con el pinout exacto de la placa ESP32-S3 física antes de soldar.

Función	Pin ESP32-S3	Voltaje	Módulo / conexión	Resistencias / notas
I2C SDA	GPIO8	3.3V lógica	OLED + IMU + PCA9685	Pull-up 4.7k a 3.3V si los módulos no lo traen.
I2C SCL	GPIO9	3.3V lógica	OLED + IMU + PCA9685	Pull-up 4.7k a 3.3V si los módulos no lo traen.
OLED actual	GPIO8/9	3.3V	SSD1306 128x64 I2C, addr 0x3C	Usar para cara anime inicial.
IMU	GPIO8/9	3.3V	MPU6050 u otra IMU I2C, addr típica 0x68	Compartir bus I2C. Evitar 5V si módulo no regula.
Servos	GPIO8/9 vía PCA9685	VCC 3.3V, V+ 5V	PCA9685 addr 0x40; CHO-CH5 servos	Capacitor 1000-2200uf en 5V servo. GND común.
I2S CLK	GPIO4	3.3V lógica	INMP441 + MAH98357A	Bus I2S compartido. Ya probado en duplex.
I2S WS/LRCK	GPIO5	3.3V lógica	INMP441 + MAH98357A	Bus I2S compartido.
I2S MIC SD	GPIO7	3.3V lógica	INMP441 SD/DOUT	Mantener cable corto y lejos de servos si hay ruido.
I2S SPK DIN	GPIO6	3.3V lógica	MAH98357A DIN	MAH98357A SD a 3.3V para habilitar.
BOOT externo	GPIO0	3.3V lógica	Botón momentáneo a GND	Pulsación corta para escuchar; mantener al reset entra bootloader. Pull-up 10k si no viene en placa.
RESET externo	EN/RST	3.3V lógica	Botón momentáneo a GND	No es GPIO. Pull-up suele venir en placa; opcional 10k a 3.3V y 100nF a GND.
Touch cabeza	GPIO1 / Touch T1	3.3V	Pad táctil	Sin resistencia normalmente. Cable corto o blindado si es sensible.
Touch izquierda	GPIO2 / Touch T2	3.3V	Pad táctil	Reservado para caricia/lado izquierdo.

Función	Pin ESP32-S3	Voltaje	Módulo / conexión	Resistencias / notas
Touch derecha	GPIO3 / Touch T3	3.3V	Pad táctil	Reservado para caricia/lado derecho.
Touch cuerpo/base	GPIO11 / Touch T11	3.3V	Pad táctil	Cuarto touch mínimo. Se mueve desde GPIO10 para dejar GPIO10 como ADC de batería.
Touch extra 1	GPIO12 / Touch T12	3.3V	Pad táctil opcional	Reservado.
Touch extra 2	GPIO20 / Touch/entrada opcional	3.3V	Pad táctil opcional	Solo si el pin está disponible en la placa. Mantener como reserva.
PIR	GPIO13	3.3V entrada	Mini PIR OUT	Si OUT es 5V, usar divisor: 10k arriba y 20k abajo aprox.
Pantalla verde futura SPI CLK	GPIO14	3.3V	TFT/OLED SPI futura	Reservado; no soldar hasta elegir pantalla.
Pantalla verde futura SPI MOSI	GPIO15	3.3V	TFT/OLED SPI futura	Reservado.
Pantalla verde futura CS	GPIO16	3.3V	TFT/OLED SPI futura	Reservado.
Pantalla verde futura DC	GPIO17	3.3V	TFT/OLED SPI futura	Reservado.
Pantalla verde futura RST	GPIO18	3.3V	TFT/OLED SPI futura	Reservado.
Pantalla verde futura BL	GPIO21	3.3V/PWM	Backlight si aplica	Con transistor si el backlight consume más de lo permitido.
Cámara futura	Por definir	3.3V/5V según módulo	Preferir ESP32-S3-CAM separado o Pi/USB	Una cámara paralela consume muchos pines; no fijar aún.
Medición batería VBAT	GPIO10 / ADC1	Entrada ADC máx. 3.3V	Divisor resistivo desde batería 1S antes del boost	R1 100k entre VBAT y ADC; R2 100k entre ADC y GND; capacitor 100nF ADC-GND. Medir 0-2.1V para 0-4.2V. GND común.

Asignación sugerida de servos en PCA9685

Canal PCA9685	Movimiento	Uso expresivo	Notas de seguridad
CH0	Cuello yaw	Mirar izquierda/derecha	Calibrar centro y límites físicos.
CH1	Cuello pitch	Mirar arriba/abajo, asentir	Evitar topes al iniciar.
CH2	Cintura/base	Giro corporal suave	Opcional si el cuerpo lo permite.
CH3	Brazo izquierdo	Saludo, emoción, gesto tímido	Movimiento lento.
CH4	Brazo derecho	Saludo, énfasis al hablar	Movimiento lento.
CH5	Extra / brazo roll / accesorio	Expresión secundaria	No usar hasta tener fuente estable.

4. Alimentación y batería

La batería sigue pendiente. Por ahora se recomienda desarrollar con powerbank o fuente 5V externa para separar problemas de firmware de problemas de energía.

Opción	Ventaja	Riesgo	Recomendación
Powerbank 20.000 mAh	Muchísima autonomía para pruebas; fácil USB.	Puede apagarse o limitarse con picos de servos.	Usar en banco. Revisar salida 5V 2A/3A.
LiPo 1S 2000 mAh	Compacta, entra más fácil.	Autonomía menor; IC puede limitar servos.	Buena para prototipo liviano si cabe.
LiPo 1S 5000 mAh	Mejor autonomía, cerca de 18.5 Wh.	Puede ser grande; verificar protección y corriente.	Mejor candidata si cabe sin apretar.
2S + buck 5V	Mejor para corriente y eficiencia.	Más volumen y carga más compleja.	Evaluar solo si 1S no aguanta picos.

- Usar 5V separado para servos y GND común con ESP32-S3.
- No alimentar servos desde 3.3V ni desde la placa ESP32.

- Agregar capacitor de 1000uf a 2200uf en la línea 5V de servos.
- Botón de emergencia ideal: cortar alimentación de servos, no apagar toda Hi.
- Medir consumo real por etapas antes de elegir batería final.

4.1 Medición de estado de batería

Se agrega una medición de batería porque Hi puede usar su energía como parte de su lenguaje corporal: cara cansada, voz más suave, aviso de carga baja, reducción de movimientos y corte preventivo de servos en carga crítica.

Recomendación inicial: medir la batería cruda 1S, es decir VBAT antes del boost de 5V. Así la lectura representa el estado real de la LiPo y no la salida regulada.

- GPIO recomendado: GPIO10 como ADC1. Se libera moviendo el touch cuerpo/base a GPIO11.
- Divisor recomendado para LiPo 1S: R1 100k desde VBAT al ADC y R2 100k desde ADC a GND. Con 4.2V de batería, el ADC ve aproximadamente 2.1V, seguro para 3.3V.
- Agregar un capacitor de 100nf entre ADC y GND para estabilizar la lectura.
- No conectar VBAT directo al ADC. La batería llena llega a 4.2V y puede dañar el pin.
- La lectura debe calibrarse por software con multímetro: voltaje real de batería versus valor leído por ADC.
- Estados sugeridos: 4.10V llena; 3.85V normal; 3.60V baja; 3.40V crítica; 3.20V apagar servos y pedir carga. Ajustar después de medir bajo carga.
- Cuando la batería esté baja, Hi debe reducir gestos, bajar brillo si aplica, evitar movimientos simultáneos y avisar con cara cansada o preocupada.

5. Plan de trabajo desde lo fácil a lo complejo

Fase	Objetivo	Tareas principales	Criterio de listo
0	Congelar versión viva	Guardar repo/branch estable. Documentar MiaoZhi S3 fundacional.	Se puede volver al firmware que habla y conecta.
1	Banco de energía seguro	Powerbank/fuente 5V. GND común. Capacitor. Sin batería interna.	S3 no se reinicia al hablar.
2	PCA9685 + 1 servo	Probar I2C, mover un servo lento, calibrar centro.	Un servo se mueve sin ruido ni reset.
3	6 servos en banco	Agregar servos uno por uno. Secuencia suave.	Gestos básicos sin caídas de voltaje.
4	Botones externos	Cablear Reset y BOOT a botones de carcasa.	Se puede escuchar/resetear sin abrir cuerpo.
5	Touch mínimo 4 pads	Cabeza, izquierda, derecha, cuerpo. Ajustar sensibilidad.	Touch cabeza activa escucha o evento.
6	IMU + PIR	IMU en I2C, PIR en GPIO13. Eventos de movimiento/presencia.	Hi detecta inclinación y presencia.
7	Cara anime OLED	Motor ojo izq + ojo der + boca. Estados: idle, listening, speaking, happy, sad, angry.	Hi muestra emoción clara.
8	Integración cuerpo	Meter S3, cableado, servos y sensores en cuerpo de resina.	Cuerpo tierra sin apretar batería/cables.
9	Servidor local	Levantar xiaozhi-esp32-server. Probar context provider y Ollama/TTS.	Hi responde con datos del HUB/local.
10	Voz emocional Nlya/Hi	Probar ElevenLabs v3 y ruta local. Crear dataset/voz propia.	Hi/Nlya tienen voz consistente.
11	Cámara	Elegir módulo y arquitectura. Probar captura básica.	Hi ve sin romper energía/audio.
12	Batería interna	Medir consumo real. Elegir LiPo/BMS/boost/carga.	Autonomía real comprobada.
4B	Telemetría de batería	Agregar divisor VBAT a ADC1, calibrar lectura, definir estados llena/normal/baja/crítica y mapearlos a cara OLED y seguridad de servos.	Hi avisa batería baja y cambia expresión sin reinicios ni lecturas erráticas.

6. Paquete para Codex

Objetivo para Codex: convertir el prototipo actual en un firmware jRobot mantenible, con pines documentados, cara anime, servos expresivos y camino hacia servidor local.

- Crear rama: feature/xi-electronbot-body-v02.
- No romper la versión funcional de XiaoZhi S3.
- Agregar carpeta docs/xi-electronbot-body/ con este datasheet y plan.
- Crear módulo jrobot_face: dibujo OLED de ojos y boca independientes.
- Crear módulo jrobot_motion: control seguro de servos por PCA9685.
- Crear módulo jrobot_sensors: touch, IMU y PIR.
- Agregar módulo jrobot_power: lectura ADC de batería, filtro, estados de carga y eventos expresivos/seguridad.
- Agregar configuración de pines centralizada en main/boards/jrobot-s3/config.h o equivalente.
- Agregar estados expresivos: idle, listening, speaking, thinking, happy, sad, angry, error.
- Integrar primero solo eventos visuales; después servos; después HUB/context provider.
- Mantener README con pasos de build, flash y recuperación.

Prompt sugerido para Codex

Implementar una evolución del firmware jrobot-s3 para Xi ElectronBot Body v0.3. Mantener el hardware de audio y OLED ya funcional. Agregar soporte incremental para PCA9685 en I2C compartido, seis servos expresivos con límites seguros, cuatro sensores touch, IMU I2C y PIR. Crear un motor de cara OLED anime basado en objetos independientes: ojo izquierdo, ojo derecho y boca. No activar movimientos peligrosos al arranque. Documentar pines, comandos de prueba, dependencias y checklist de validación. Trabajar en commits pequeños y dejar siempre una ruta para volver a la versión estable. Agregar medición de batería por ADC I con divisor resistivo, filtro y eventos de estado para OLED/servos.

7. Reglas de seguridad y evolución

- Primero energía estable, luego sensores, luego servos, luego batería interna.
- Los servos nunca deben moverse sin límites calibrados.
- El botón de emergencia debe cortar servos o forzar SAFE_Off.
- La batería LiPo no debe ir apretada, doblada, perforada ni encerrada sin revisión térmica.
- Xi puede tener personalidad emocional, pero las reglas de seguridad no deben mutar.
- La voz Nya/Xi debe crearse con material propio o autorizado.

Nombre de etapa sugerido: Xi Body v0.3 - Segundo nivel de hardware. Evolución: placa -> tupper -> cuerpo con sensores y expresión física.

8. Fuentes y notas

- ElevenLabs pricing/API consultado el 2026-08-27: <https://elevenlabs.io/pricing/api> y <https://elevenlabs.io/pricing>
- ElevenLabs v3/audio tags consultado el 2026-08-27: <https://elevenlabs.io/v3> y documentación de Text-to-Speech API.
- ElectronBot de referencia: <https://github.com/peng-zhihui/ElectronBot>
- XiaoZhi firmware/server de trabajo: <https://github.com/Jcabroc/xiaozhi-xi> y <https://github.com/Jcabroc/xiaozhi-esp32-server>
- Este datasheet de pines es una propuesta inicial; verificar el pinout exacto de la placa física antes de soldar.